Software Test Plan (STP)

Railway Reservation System

Software Engineering Laboratory Project, PES University

Team 11, Class 5A CSE

Version	Date	Status
1.3	11 November 2026	Submitted for review

Team

	Abhigyan Dutta	Akash Singh	Aditya Alur	Aditya G. Mule
SRN	PES2UG24CS019	PES2UG24CS042	PES2UG24CS052	PES2UG24CS031

Revision history

Ver	Date	Author	Change summary
0.1	14 Sep 2026	Abhigyan Dutta	Test plan outline, scope, test items drawn from SRS and SAD
0.2	15 Sep 2026	Akash Singh	Test environment, tools, performance and security approach
0.3	15 Sep 2026	Aditya Alur	Security validation section and risk table
1.0	16 Sep 2026	Abhigyan Dutta	Traceability completed, test case ownership assigned, plan frozen
1.1	09 Nov 2026	Abhigyan Dutta	Test case specifications added as Section 13.1 with Sprint 1 results; tracking moved to GitHub Projects; CI database and static analysis updated; typo fixes
1.2	10 Nov 2026	Abhigyan Dutta	Sprint 2 results: 15 test cases (payment, tickets, refunds, cancellation, chart, handheld, administration, security) marked Pass with their test files; totals 94 unit and 179 integration tests
1.3	11 Nov 2026	Abhigyan Dutta	Sprint 3 results: TC-PERF-01, TC-PERF-02, TC-REL-01 and TC-UX-01 marked Pass with their reports; TC-REL-02 added for RRS-NF-004 and TC-CAN-03 for defect #67 (32 test cases); totals 103 unit and 192 integration tests; four test reports in docs/reports

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof. Pavithra	
Class Coordinator	Prof. Deena Hajim	

1. Introduction

Purpose. This document defines the test plan for the Railway Reservation System v1.0, updated after Sprint 1. It sets out the objectives, scope, strategy, environment, schedule, and responsibilities for testing the system described in the SRS (v1.0) and designed in the SAD (v1.0).

Scope. Testing covers passenger registration, train and availability search, reservation with RAC, waitlist and Tatkal, fare calculation and payment, cancellation with refund and waitlist promotion, chart preparation, offline ticket verification, and administration with an audit trail. Both the passenger channel and the reservation counter are tested against the one shared inventory.

Out of scope. Payment gateway internals, the external SMS and email provider internals, train operations and signalling, and the items named out of scope in SRS Section 1.2 (segment wise berth reuse, concession fares, saved traveller profiles).

References. SRS Railway Reservation System v1.0, SAD Railway Reservation System v1.0, RTM in SRS Section 8, OWASP Top 10 (https://owasp.org/www-project-top-ten/), IEEE 829 test documentation.

Definitions. PNR (Passenger Name Record), RAC (Reservation Against Cancellation), Tatkal (late release quota), TC (Test Case), RTM (Requirements Traceability Matrix), UAT (User Acceptance Testing), p95 (95th percentile). Domain terms are defined in SRS Section 1.4.

2. Test Items

The following modules, drawn from SAD Section 3.4, are under test:

- AccountService: registration and OTP verification
- SearchService and InventoryService: train search and live availability
- InventoryService and AllocationEngine: berth holding, allocation, RAC, waitlist, Tatkal
- FareEngine and PaymentAdapter: fare computation and payment
- RefundService: cancellation refunds and auto refund on allocation failure
- ChartService and HandheldApp: chart preparation and offline ticket verification
- AdminConsole and AuditLog: schedule and fare administration, privileged action logging
- AuthService: RBAC, second factor, account lockout

3. Features to be Tested

Features mapped to SRS requirement IDs. Every High priority requirement in the SRS RTM has a test case here.

Requirement ID	Feature	Test case(s)
RRS-F-001, F-002	Registration and OTP verification	TC-REG-01, TC-REG-02
RRS-F-003, F-004	Train search and live availability	TC-SRCH-01, TC-SRCH-02
RRS-F-005, F-006	No double allocation, 10 minute hold	TC-BOOK-01, TC-BOOK-02
RRS-F-007	Quota, RAC, waitlist order	TC-BOOK-03
RRS-F-009	Tatkal opening window	TC-BOOK-05
RRS-F-010 to F-014	Fare, payment, ticket issue, idempotency, auto refund	TC-PAY-01 to TC-PAY-05
RRS-F-015, F-016	Cancellation slab refund, waitlist promotion	TC-CAN-01, TC-CAN-02
RRS-F-017, F-018	Chart preparation, offline verification	TC-CHT-01, TC-CHT-02
RRS-F-019, F-020	Schedule and fare admin, privileged audit	TC-ADM-01, TC-ADM-02
RRS-NF-001, NF-002	Booking and search latency under load	TC-PERF-01, TC-PERF-02

Requirement ID	Feature	Test case(s)
RRS-NF-003	Inventory reconciles exactly	TC-REL-01
RRS-NF-005, SR-001 to SR-005	Card data, TLS, RBAC, masking, hashing	TC-SEC-01 to TC-SEC-06

4. Features Not to be Tested

- Payment gateway card processing (vendor responsibility; the system only initiates and reconciles)
- SMS and email provider delivery internals (external service; tested via a mock)
- Train schedule feed accuracy (assumed loaded and correct before booking opens)
- Two Medium priority functional requirements (RRS-F-008 passenger limit, RRS-NF-004 monthly availability) are tracked in the team repository, not in this plan

5. Test Approach and Strategy

Levels.

- Unit tests: individual module logic (fare math, slab lookup, quota decision) with Jest
- Integration tests: module to module and module to database, against a real PostgreSQL (a service container in GitHub Actions, Docker Compose locally), and Supertest for Express routes
- System tests: end to end booking, cancellation, chart and verification flows
- Acceptance tests: the exit criteria in SRS Section 6, checked against the acceptance suites

Types.

- Functional testing of every High priority requirement
- Concurrency testing of the allocation path (200 concurrent requests against limited berths)
- Performance testing with k6 (50 booking attempts per second, 1000 concurrent searches)
- Reliability testing (inventory reconciliation after a simulated Tatkal burst)
- Security testing (TLS scan, card data scan, RBAC, PNR masking, password hashing)
- Usability and accessibility testing (WCAG 2.1 AA, throttled connection run)

Entry criteria. A stable build is deployed to the test environment, test data is loaded, and the database is at a known clean state.

Exit criteria. Every High priority test case is executed, no critical or high severity defect is open against the allocation, payment or cancellation paths, and every High priority requirement in the RTM shows a passing test case.

5.1 Security Validation

Drawn from SAD Section 3.9 (STRIDE threat model) and the SRS security requirements:

- Card data handling: scan database, logs and traffic captures for card numbers or verification codes (must find none)
- TLS: confirm TLS 1.2 or higher on every connection, plain HTTP redirected
- RBAC and least privilege: a clerk account cannot reach the administration console, second factor enforced for clerk and admin
- PNR masking: unauthenticated lookup returns initials only, rate limited to 30 per minute
- Password storage: memory hard hash with per account salt, account lockout after 5 failed attempts
- Fuzz testing of input fields on the booking and payment forms
- Penetration testing of the authentication and payment callback flows

6. Test Environment

Item	Detail
Server	Linux host, Docker Compose (Node.js app container + PostgreSQL 15 container)
Application	Railway Reservation System v1.0 (Node.js 20, Express)
Database	PostgreSQL 15 with serialisable isolation on inventory tables, seeded with test data
Handheld	Android Chrome for the PWA, with the network disabled to test offline verification
Payment	Mock gateway that returns success, decline and repeated callback on demand
Tools	Jest (unit, with line and branch coverage), Supertest (API), PostgreSQL service in GitHub Actions (integration), k6 (load), GitHub Issues and Projects (defects and tracking), ESLint (static analysis)
Test data	Dummy passenger accounts, three sample trains per station pair, a coach with a known berth count, fare and slab configuration

7. Test Schedule

Milestone	Date
Test case design complete	18 Sep 2026
Test environment setup	20 Sep 2026
Test execution start	22 Sep 2026
Test execution end	03 Oct 2026
Performance and reliability run	04 Oct 2026
UAT	06 Oct 2026 to 09 Oct 2026

The suites run in dependency order (Registration, Search, Booking, Tatkal, Payment, Cancellation, Chart and Checking, Administration, Performance, Reliability, Security, Accessibility) because each suite uses data the previous one creates. Performance runs last on a restored copy so a load run cannot corrupt functional test data.

8. Test Deliverables

- Test Plan (this document)
- Test cases (manual and automated)
- Test scripts (Jest, Supertest, k6)
- Test data set and seed scripts
- Test execution logs
- Defect reports (GitHub Issues)
- Static analysis and coverage reports (ESLint and Jest coverage, run in CI)
- Test summary report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Abhigyan Dutta	Prepare and maintain the test plan, coordinate execution, own the traceability matrix, run the final acceptance check
Test Engineer	Aditya G. Mule	Design and execute assigned test cases, log defects, maintain test data

Role	Name	Responsibility
Test Engineer	Akash Singh	Design and execute assigned test cases, log defects, maintain test data
Test Engineer	Aditya Alur	Design and execute assigned test cases, log defects, maintain test data
Product Owner (sign-off)	Abhigyan Dutta	Approve test results and sign off release readiness

Each team member owns seven High priority test cases, listed in Section 13. Defect fixes are handled by whoever owns the module under the SAD component ownership, with review by the QA Lead.

10. Risks and Mitigation

Risk	Mitigation
Unstable build blocks test execution	Request early smoke builds; run the functional suite against each nightly build
Test environment downtime	Docker Compose lets any team member bring the environment up locally in minutes
Mock gateway does not reproduce real callback timing	Script the mock to send delayed and repeated callbacks to exercise idempotency and hold expiry
Concurrency test is flaky (timing dependent)	Run TC-BOOK-01 repeatedly in CI; require a consistent pass across 10 runs before acceptance
Load test corrupts functional data	Performance suite runs last on a restored database copy

11. Assumptions and Dependencies

- The mock payment gateway is available and configurable for success, decline and repeat callbacks
- Test data (accounts, trains, coach composition, fare and slab rules) is loaded before execution
- The train schedule feed is loaded before booking opens for a test date
- Server clocks are synchronised, since Tatkal opening and cancellation cutoff are decided on server time

12. Suspension and Resumption Criteria

Suspend testing if:

- The test environment is unavailable for more than 4 hours
- A build is too unstable and blocks more than 30 percent of planned test cases
- A critical defect on the allocation or payment path blocks dependent suites

Resume testing if:

- Blocking defects are resolved and verified
- The environment is stable and the database is restored to a known state

13. Test Case Management and Traceability

The RTM in SRS Section 8 maps every requirement to a test case. This section adds the owner for each High priority test case. Each of the four team members owns seven cases.

Requirement ID	Test case	Owner
RRS-F-001	TC-REG-01	Aditya G. Mule
RRS-F-002	TC-REG-02	Aditya G. Mule

Requirement ID	Test case	Owner
RRS-F-003	TC-SRCH-01	Aditya G. Mule
RRS-F-004	TC-SRCH-02	Aditya G. Mule
RRS-F-005	TC-BOOK-01	Abhigyan Dutta
RRS-F-006	TC-BOOK-02	Abhigyan Dutta
RRS-F-007	TC-BOOK-03	Abhigyan Dutta
RRS-F-009	TC-BOOK-05	Aditya Alur
RRS-F-010	TC-PAY-01	Akash Singh
RRS-F-011	TC-PAY-02	Akash Singh
RRS-F-012	TC-PAY-03	Abhigyan Dutta
RRS-F-013	TC-PAY-04	Abhigyan Dutta
RRS-F-014	TC-PAY-05	Akash Singh
RRS-F-015	TC-CAN-01	Aditya Alur
RRS-F-016	TC-CAN-02	Aditya Alur
RRS-F-017	TC-CHT-01	Aditya Alur
RRS-F-018	TC-CHT-02	Aditya G. Mule
RRS-F-019	TC-ADM-01	Aditya G. Mule
RRS-F-020	TC-ADM-02	Aditya G. Mule
RRS-NF-001	TC-PERF-01	Abhigyan Dutta
RRS-NF-002	TC-PERF-02	Akash Singh
RRS-NF-003	TC-REL-01	Abhigyan Dutta
RRS-NF-005	TC-SEC-01	Akash Singh
RRS-SR-001	TC-SEC-02	Akash Singh
RRS-SR-002	TC-SEC-03	Akash Singh
RRS-SR-003	TC-SEC-04	Aditya Alur
RRS-SR-004	TC-SEC-05	Aditya Alur
RRS-SR-005	TC-SEC-06	Aditya Alur

Sample test case mapping detail:

- RRS-F-005 (no double allocation) to TC-BOOK-01: 200 concurrent requests against 10 free berths, expect exactly 10 confirmations and no repeated berth
- RRS-F-013 (idempotent callback) to TC-PAY-04: the same callback delivered five times results in one PNR and one charge
- RRS-NF-001 (booking latency) to TC-PERF-01: 50 booking attempts per second, p95 under 3 seconds with no failed inventory write

13.1 Test Case Specifications

Each test case below gives its input and steps, the expected result, the level it runs at and its status. Sprint 1 cases are automated and run by CI on every push and pull request (50 unit and 79 integration tests at the end of Sprint 1; 94 unit and 179 integration tests at the end of Sprint 2; 103 unit and 192 integration tests at the end of Sprint 3); the file named is where each case is implemented. Sprint 2 cases were added to CI as each feature was merged. Sprint 3 added the load, reliability, availability and accessibility cases and TC-REL-02 for RRS-NF-004. The four test reports (unit, system validation, code coverage, branch coverage) and their evidence are in docs/reports in the repository.

Test case	Input and steps	Expected result	Level and status
TC-REG-01 RRS-F-001, functional	Register a new passenger; register again with the same mobile and a different email; try 9 and 11 digit mobiles.	First: 201, unverified. Second: 409 MOBILE_TAKEN naming the mobile field. Invalid mobile: 400. Password stored only as a salted scrypt hash.	Unit and integration Sprint 1: Pass (CI) accounts.test.js
TC-REG-02 RRS-F-002, functional and security	Request an OTP; verify with the correct code, a wrong code, a code expired by 1 second, and with no code requested.	Correct code verifies. Wrong: 400, still unverified. Expired: 410. Expiry is 10 minutes. An unverified account cannot proceed to payment.	Unit and integration Sprint 1: Pass (CI) accounts.test.js
TC-SRCH-01 RRS-F-003, functional	Search SBC to MAS on a weekday and a weekend; KPD to MAS; MAS to SBC; a date past the 30 day window; an unknown station.	Weekday: 3 trains with correct times. Weekend: 12658 absent. KPD to MAS: 142 km. Wrong direction and past window: empty. Unknown station: 400.	Unit and integration Sprint 1: Pass (CI) search.test.js
TC-SRCH-02 RRS-F-004, functional	Read availability for a fresh run; hold one berth and read again; fill the berths; release a hold.	Fresh: 8 confirmed, 2 RAC, 0 waitlist. Count falls by one at once (within 30 seconds). RAC then waitlist move. A released hold restores the count.	Integration Sprint 1: Pass (CI) availability.test.js
TC-BOOK-01 RRS-F-005, functional (safety)	Send 200 hold requests at the same time, one passenger each, to one run, Sleeper, general quota.	Every free berth (8 in the demo coach) is confirmed exactly once; no berth number repeats; the inventory shows 8 held berths under 8 distinct holds.	Integration (concurrency) Sprint 1: Pass (CI) holds.test.js
TC-BOOK-02 RRS-F-006, functional	Hold all free berths; request again; let the first hold lapse; run the sweeper; release a hold twice.	Hold expires 10 minutes after creation. A held berth is not offered; it is offered again after the lapse. Release frees the berth at once and is idempotent.	Integration Sprint 1: Pass (CI) holds.test.js
TC-BOOK-03 RRS-F-007, functional	Send single passenger requests one after another: free berths, plus 2, plus 13. Book 5 passengers when 2 berths are left.	Requests get CNF, then RAC 1 and RAC 2, then WL 1 to WL 13, in order. The 5 passenger booking splits into CNF, CNF, RAC 1, RAC 2, WL 1.	Unit and integration Sprint 1: Pass (CI) allocation.test.js
TC-BOOK-04 RRS-F-008, functional (Medium)	Tatkal booking with 5 passengers and with 4; general booking with 7.	5 Tatkal and 7 general: 400 TOO_MANY_PASSENGERS. 4 Tatkal: accepted.	Integration Sprint 1: Pass (CI) tatkal.test.js
TC-BOOK-05 RRS-F-009, functional	Tatkal request for a journey 4 days away; opening time set to 00:00 and to 23:59 for tomorrow; general quota on the same run.	Early: 409 stating "opens at 10:00 on <date>". After opening: accepted. Time is read from settings. General quota unaffected. 09:59 closed and 10:00 open (unit).	Unit and integration Sprint 1: Pass (CI) tatkal.test.js
TC-PAY-01 RRS-F-010, functional	Fare for SBC to MAS in SL and 3A; KPD to MAS in SL; 199, 200 and 201 km; three sample journeys; a	SL Rs 200.00; 3A Rs 513.75; KPD to MAS Rs 125.00 (200 km minimum). Parts add up to the	Unit and integration Sprint 1: Pass (CI) fares.test.js

Test case	Input and steps	Expected result	Level and status
	fare rule dated in the future.	total. A future rule does not change today's fare.	
TC-SEC-06 RRS-SR-005, security	Sign in correctly; 5 wrong passwords then the right one; 4 wrong then right; after the lock window; an unknown mobile.	200 on success. Sixth attempt: 423 ACCOUNT_LOCKED. 4 failures do not lock. Unlocks after 15 minutes. Unknown mobile: 401 with the same message. Hashes salted and not reversible.	Unit and integration Sprint 1: Pass (CI) login.test.js
TC-UX-01 RRS-NF-006, non functional (Medium)	axe-core scan of every page in light, dark and mobile, including open pickers; keyboard only booking path; booking flow on a 1 Mbps connection.	Zero WCAG 2.2 AA violations; visible focus and logical order; booking completes within 30 seconds of transfer time.	Automated scan and manual Sprint 3: Pass docs/reports/ accessibility.md, manual-checks.md
TC-PAY-02 RRS-F-011, security	Complete a payment through the mock gateway; inspect database and logs.	No card number or verification code anywhere; only a transaction reference and a token.	Integration and scan Sprint 2: Pass (CI) payments.test.js
TC-PAY-03 RRS-F-012, functional	Successful, declined and abandoned payments on a held booking.	Success: one PNR with QR and one ticket message. Failure: no PNR and the held berths return to availability.	Integration Sprint 2: Pass (CI) bookings.test.js
TC-PAY-04 RRS-F-013, functional	Deliver the same signed gateway callback 5 times.	Exactly one PNR and one charge.	Integration (concurrency) Sprint 2: Pass (CI) bookings.test.js
TC-PAY-05 RRS-F-014, functional	Payment succeeds after the hold has lapsed and the berths are gone.	A full refund record with a reference; the passenger is notified; no PNR.	Integration Sprint 2: Pass (CI) refunds.test.js
TC-CAN-01 RRS-F-015, functional	Cancel either side of two refund slab boundaries; cancel after the cutoff.	Refunds match the slab amounts; a request after the cutoff is refused and names the cutoff.	Unit and integration Sprint 2: Pass (CI) cancellation.test.js
TC-CAN-02 RRS-F-016, functional	Cancel one confirmed berth while RAC and waitlisted passengers are queued.	Earliest RAC moves to the berth, earliest waitlisted moves to RAC, in queue order; two notifications sent.	Integration Sprint 2: Pass (CI) cancellation.test.js
TC-CAN-03 RRS-F-015, functional (regression, defect #67)	Book two passengers; cancel; book the freed berths again; open the chart, the public PNR status and the ticket for the cancelled PNR.	The cancelled passengers show Cancelled with no berth and no queue position in all three views; no berth is listed twice on the chart; the new booking is unchanged.	Unit and integration Sprint 3: Pass (CI) cancelledViews.test.js
TC-CHT-01 RRS-F-017, functional	Run chart preparation at the configured hour before departure.	Every confirmed ticket shows coach and berth; no further allocation is accepted; each remaining waitlisted ticket has a refund record.	Integration Sprint 2: Pass (CI) chart.test.js

Test case	Input and steps	Expected result	Level and status
TC-CHT-02 RRS-F-018, functional	On the handheld with the network off, check a valid ticket, a cancelled ticket and a ticket for another train.	Valid ticket: valid. Cancelled and other train: invalid with the reason.	Unit, integration and manual Sprint 2: Pass (CI) tte.test.js; offline run checked in Chrome
TC-ADM-01 RRS-F-019, functional	Add a fare rule with a future effective date; book today and after that date.	Today's booking unaffected; a booking after the date uses the new rule.	Integration Sprint 2: Pass (CI) admin.test.js
TC-ADM-02 RRS-F-020, security	Perform each of the 5 privileged action types; try to edit and delete a log row.	Each action is logged with actor, target and time; editing or deleting a row fails.	Integration Sprint 2: Pass (CI) audit.test.js
TC-SEC-01 RRS-NF-005, non functional	After payment tests, scan the database, logs and captured traffic.	No card number or verification code found.	Integration and CI scan Sprint 2: Pass (CI) cardData.test.js
TC-SEC-02 RRS-SR-001, security	TLS scan of every endpoint; request over plain HTTP.	No connection below TLS 1.2; plain HTTP is redirected.	Integration (real TLS handshakes) Sprint 2: Pass (CI) tls.test.js
TC-SEC-03 RRS-SR-002, security	Scan stores, logs and backups.	No card data in any of them.	Integration and CI scan Sprint 2: Pass (CI) cardData.test.js
TC-SEC-04 RRS-SR-003, security	A clerk opens the admin console; clerk signs in with one factor; leave a counter session idle 5 minutes.	Admin console refused; single factor sign in refused; the idle session locks.	Unit and integration Sprint 2: Pass (CI) staff.test.js
TC-SEC-05 RRS-SR-004, security	Unauthenticated PNR status lookup; 31 lookups in one minute from one client.	Initials only, no age; the 31st lookup is refused.	Unit and integration Sprint 2: Pass (CI) pnrLookup.test.js
TC-PERF-01 RRS-NF-001, non functional	k6 load test: 50 booking attempts per second for one minute from idle.	95th percentile under 3 seconds; no failed inventory write.	Performance (k6) Sprint 3: Pass docs/reports/load-booking.md
TC-PERF-02 RRS-NF-002, non functional	k6 load test: 1000 virtual users searching.	95th percentile under 2 seconds.	Performance (k6) Sprint 3: Pass docs/reports/load-search.md
TC-REL-01 RRS-NF-003, non functional	Run the reconciliation job after the Tatkal burst test.	Zero mismatched berths between inventory and issued tickets.	Reliability (k6 burst and reconciliation) Sprint 3: Pass load-tatkal-burst.md, reconciliation.md, reconcile.test.js
TC-REL-02 RRS-NF-004, non functional	Run the service with one availability sample a minute; produce the monthly report for a	The report counts the monitored minutes, lists the gap as an outage with its	Unit and integration Sprint 3: Pass (CI) availability-

Test case	Input and steps	Expected result	Level and status
	window that includes a 6 minute gap; check the 99.5 percent boundary.	length, and compares the percentage with 99.5; exactly 99.5 meets the target and one minute less does not.	report.test.js

14. Test Metrics and Reporting

Metrics collected:

- Percentage of test cases executed
- Percentage passed and failed
- Defect density (defects per module)
- Defect aging (time from raised to resolved)
- Requirement coverage (percentage of High priority requirements with a passing test)

Reports:

- Daily execution status during the execution window
- CI run per build: lint, unit and integration results with line and branch coverage
- Final test summary report at UAT sign-off

15. Approvals

Role	Name	Signature / Date
QA Lead	Abhigyan Dutta	
Course Coordinator	Prof. Pavithra	
Class Coordinator	Prof. Deena Hajim	

End of document. Version 1.3, Team 11.